

9. Software Architecture

Based on the SRS for the Course Support & Activity Dashboard (Left4Dead2), the recommended architecture is a **Modular Monolithic Architecture** using the **Model-View-Controller (MVC)** pattern. This is a single deployable application internally organized into clearly separated modules and layers, avoiding the operational overhead of microservices while preventing the code tangling of a flat monolith.

Rationale

The project supports collaborative course management with distinct user roles (Lecturer, TA/university staff, and Student) and real-time communication needs for staff. However, as a student group project handling a single university course's activity tracking, announcements, and staff coordination, a full microservices architecture would introduce unnecessary complexity (such as service discovery and inter-service network management).

An MVC modular approach allows a clean separation of concerns:

- **Presentation (View):** Delivers role-appropriate user interfaces for Students, TAs, University staff, and Lecturers.
- **Controller:** Manages request routing, permission checks, and membership validation (Verify Classroom Membership and Role).
- **Model / Business Logic:** Encapsulates business rules such as risk-status evaluation (Overdue, Due Soon, Unassigned), classroom join-code generation, and role-promotion constraints.

Component	Layer/Type	Description
Web/Mobile Client	Presentation (View)	Renders the class schedule, dashboards, announcement boards, and file upload forms tailored to the active user's role (Student, TA, or Lecturer).
Classroom and Activity Controller	Controller	Handles HTTP requests for classroom setup, role management, activity creation, risk-status filtering, and message posting; performs authorization checks before invoking business logic

Course Model / Business Logic	Model	Encapsulates core rules including unique join-code validation, role-permission rules, and automatic risk classification algorithms (Overdue, Due Soon, Unassigned).
Course Database	Persistence	Stores structured relational data for users, memberships, roles, activities, announcements, staff messages, and file submission metadata.
Authentication & Role Module	Cross-cutting	Verifies classroom membership and checks whether a user holds Student, TA, or Lecturer privileges for every secured request.

10. Data Storage Technology

The recommended data storage technology is a Relational Database Management System (RDBMS), specifically **MySQL**, used as the primary data store for this iteration, together with a separate file store for the files students upload.

Rationale

- Nearly all data described in the SRS is structured and fits a fixed schema: users, classrooms, memberships and roles, activities, announcements, staff messages, and submission records, each with well-defined attributes and clear relationships to one another. There are no nested or variable-shape structures in this iteration that would justify a document-oriented or key-value NoSQL store. MySQL additionally enforces data integrity constraints directly at the storage layer, such as the unique join code required by SRS-2, the mandatory fields of SRS-1, SRS-9, and SRS-16, and the fixed value sets for role, status, and visibility (SRS-5, SRS-11, SRS-14), and it supports transactional consistency for the multi-row operations of SRS-6, SRS-12, and SRS-23. Its indexed ordering and date comparison also answer the dashboard, at-risk, announcement, and submission-count queries of SRS-13, SRS-17, SRS-20, and SRS-25 in a single query rather than in application code. MySQL is accessed from the Node.js and Express backend through the mysql2 connection pool. A second store is required because SRS-21 to SRS-25 introduce uploaded files, the only unstructured data in this system. Storing file bytes in MySQL as BLOB columns would inflate every database dump and slow the queries that share the same storage, so the database keeps the submission record and a reference, while the file store keeps the bytes.

Component	Data Storage Technology	Description
Course & Activity Database	MySQL 8 (InnoDB), accessed via the mysql2 pool	Single relational schema storing tables for users, classrooms, memberships, activities, announcements, staff channel messages, and student submission metadata; supports the queries and constraints required by SRS-1 through SRS-25.
Submission File Storage	Docker volume on the application host	Stores the bytes of files uploaded by students, keyed by the submission record in MySQL. Written only after the size and type check of SRS-22, and read only through an authorised Express route (SRS-25).

Only these two stores are required for the current SRS scope. Should future iterations introduce less-structured data, such as activity logs or notification payloads, a supplementary NoSQL document store could be added at that time without disrupting the relational schema.

11. Development Environment

The recommended development and deployment environment is a **containerized environment (using Docker and Docker Compose)** rather than bare metal.

Rationale

The system must run consistently across four team members' laptops and on whatever machine the department uses for the demonstration. Docker packages the Node.js and Express application, its npm dependencies, and the MySQL server into isolated containers described once in a Compose file, so every developer runs the same runtime and database version with a single command. This matters directly for the requirements: the at-risk classification of SRS-12 and the late-submission rule of SRS-24 depend on the environment's clock and time zone, and the ordering rules of SRS-17 and SRS-20 depend on how MySQL stores and compares DATETIME values, so pinning the images removes a class of defect that would otherwise appear on one machine only. Containers also make it simple to drop and re-seed the database with dated test data while testing risk classification and re-submission, without touching a developer's own MySQL installation. Bare-metal deployment would tie the system to one machine's installed Node.js version, MySQL version, and file paths, and would leave the uploaded files of Section 10 in a loose host directory rather than a declared, backed-up volume.

Component	Development Environment	Description
Application Server (View, Controller, Model, Auth & Role middleware)	Docker container running Node.js with Express	Runs the MVC application logic in an isolated, reproducible image and serves the HTML, CSS, and client-side JavaScript to the browser; deployed identically on a developer laptop or the demonstration host.
Database Server	MySQL container (via Docker Compose) with a named volume	Runs MySQL in its own container, orchestrated alongside the application container, keeping data storage decoupled from the host machine's configuration and allowing the schema to be reset and re-seeded from versioned SQL scripts.

Submission File Storage	Docker volume mounted into the application container	Holds uploaded submission files outside the container's writable layer so they survive a rebuild and can be backed up separately from the database dump.
Local Development Machine	Docker Desktop, with Node.js installed for editor tooling	Used by each team member to build and test the full stack before deployment, mirroring the production container configuration to minimise environment inconsistencies.

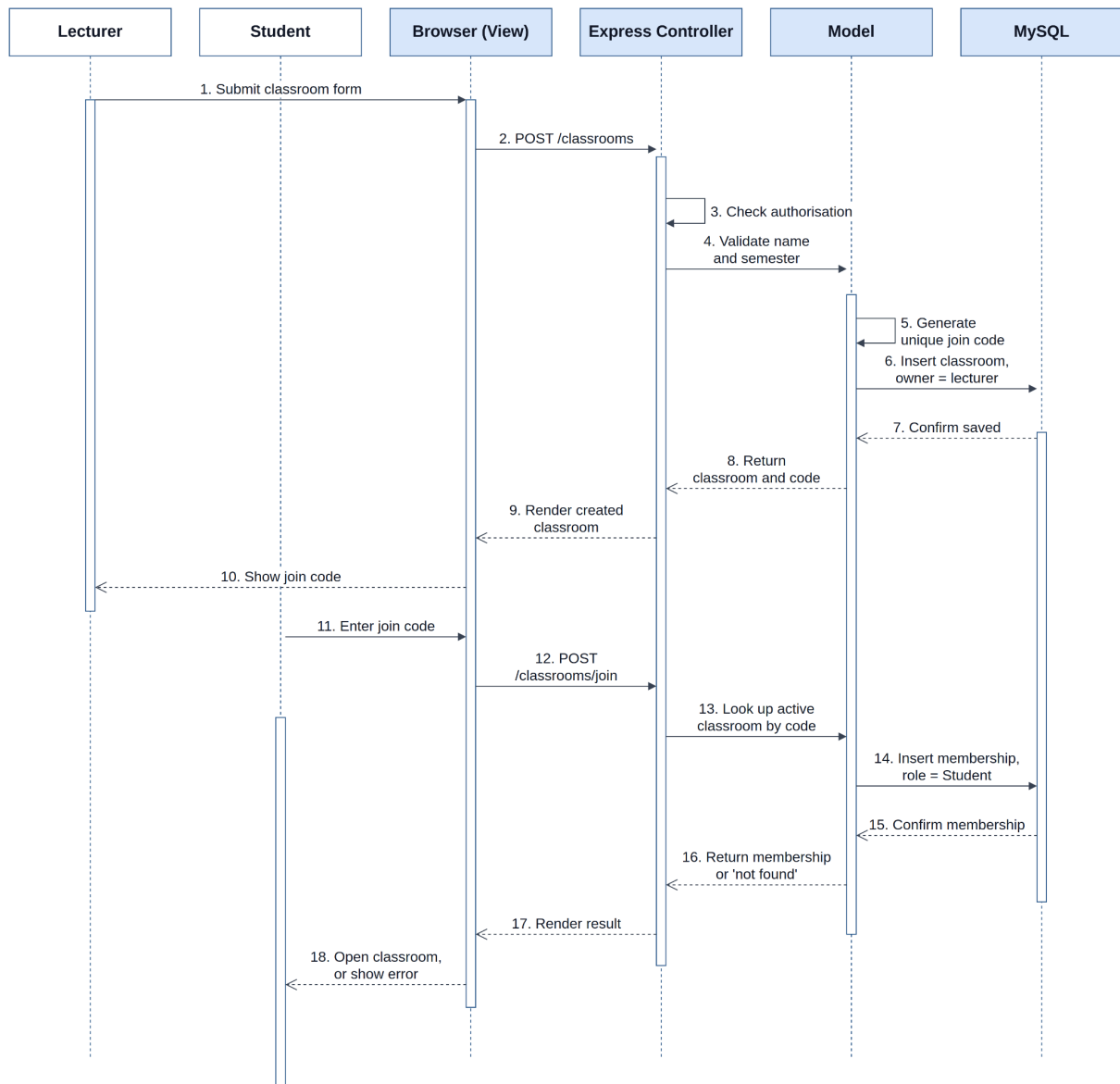
12. Sequence Diagrams for All SRS

The following sequence diagrams are derived from the chosen MVC modular architecture (Section 9) and cover all five sub-goals traced in the traceability matrix: SG-1 classroom membership and role-based access, SG-2 course activity records, SG-3 the integrated role-appropriate view, SG-4 at-risk work and unmet deadlines, and SG-5 classroom communication. Between them, they account for every requirement from SRS-1 to SRS-25 and all sixteen use cases of Section 4.

Each participant is one of the components named in Section 9: the Browser as the View, the Express Controller, the Auth and Role Middleware, the Model, the MySQL database, and the File Storage introduced in Section 10. A solid arrow is a call, and a dashed arrow is a return. A self-call is work performed inside one component. An alt fragment shows two mutually exclusive outcomes, and a loop fragment shows repetition. UC-15 Validate Activity Details and UC-16 Verify Classroom Membership and Role have no diagram of their own because they are included use cases; they appear inside the diagrams of the use cases that include them. SRS-7 is realised by the Auth and Role Middleware being consulted on each request rather than at join time, which is why that check recurs at the start of every protected interaction instead of appearing once.

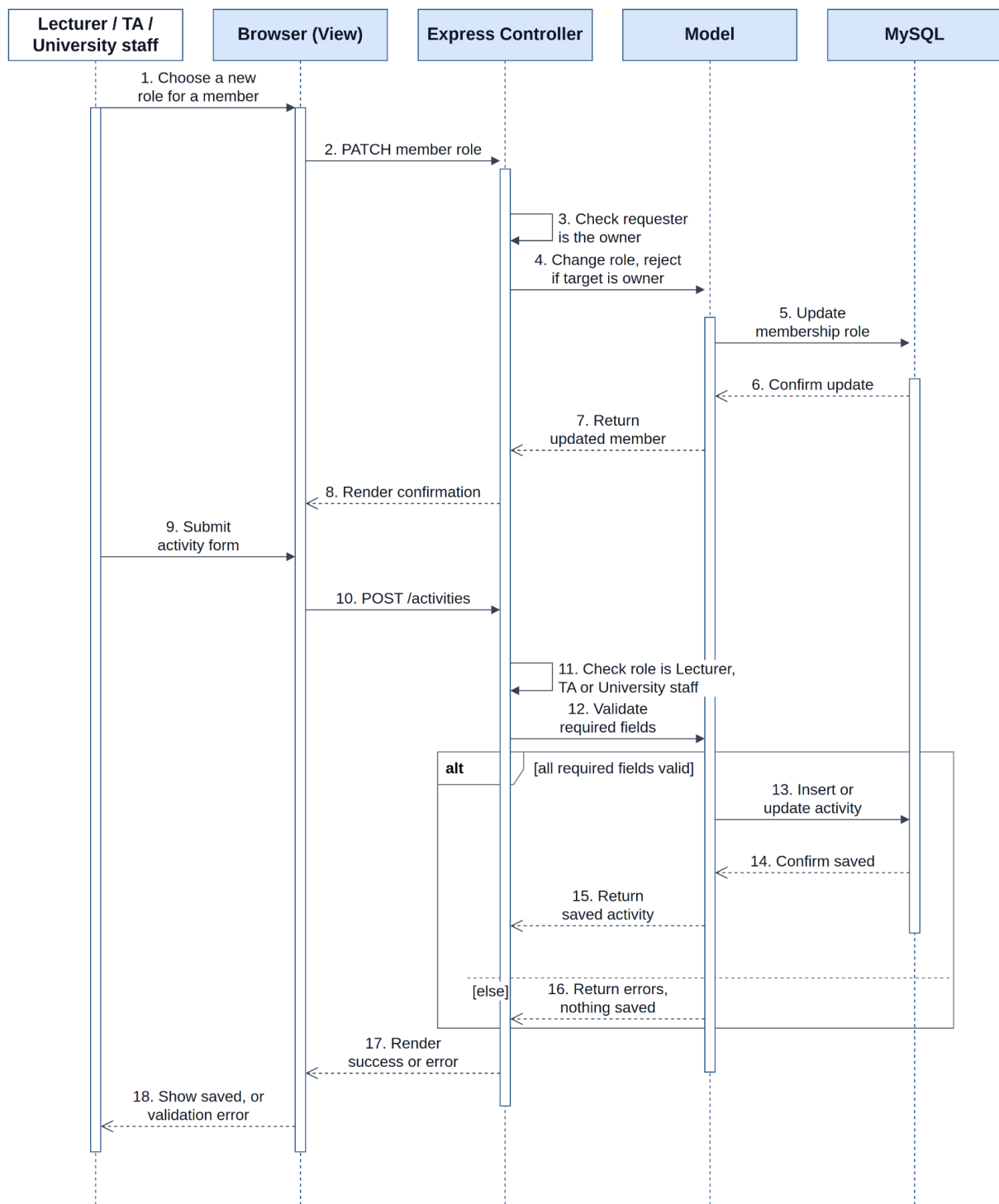
SQD-1: Classroom Setup, Membership and Role Assignment

Traced requirements: SRS-1, SRS-2, SRS-3, SRS-4, SRS-5, SRS-6, SRS-7



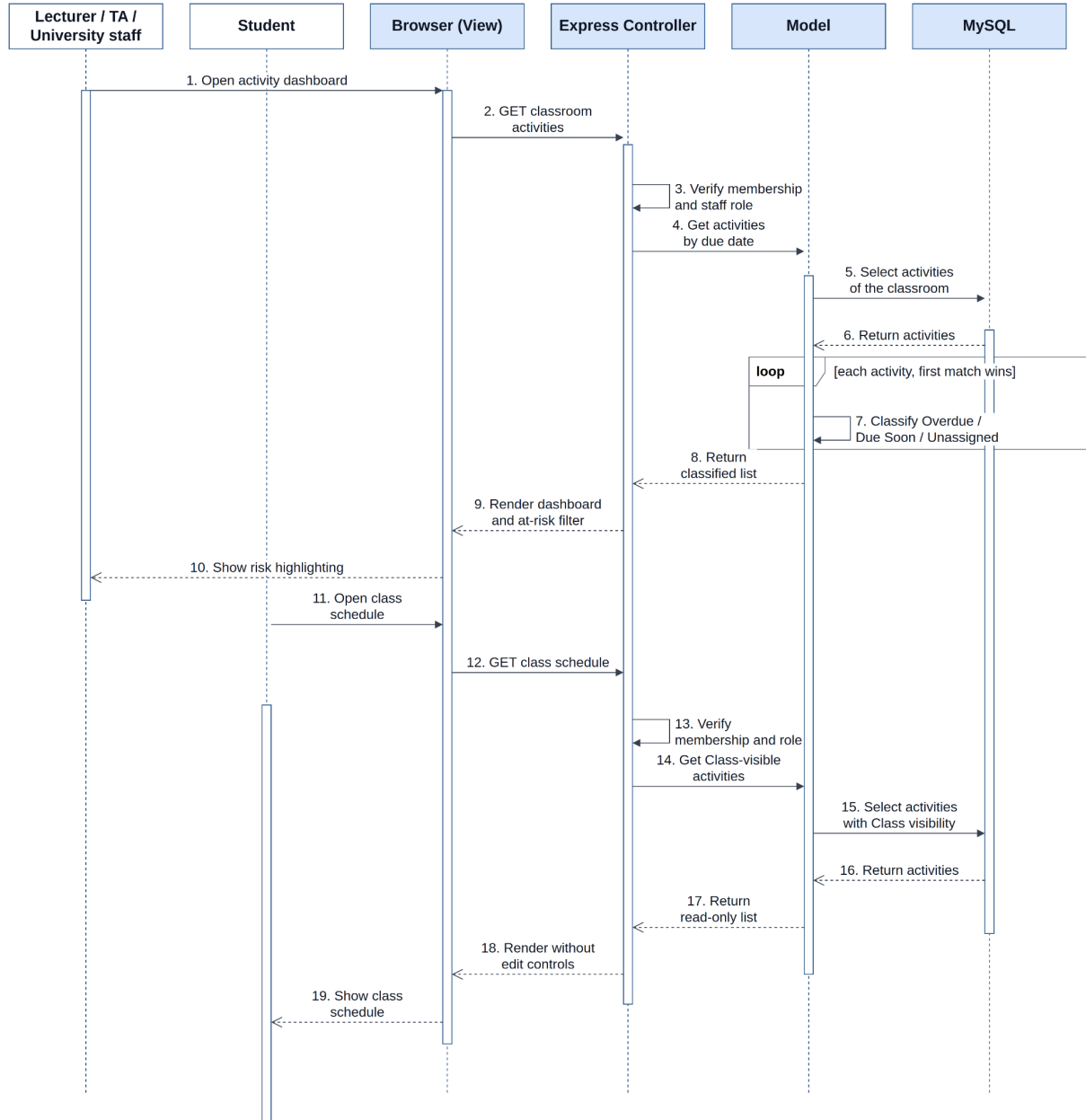
SQR-2: Staff Creates or Edits an Activity

Traced requirements: SRS-8, SRS-9, SRS-10, SRS-11



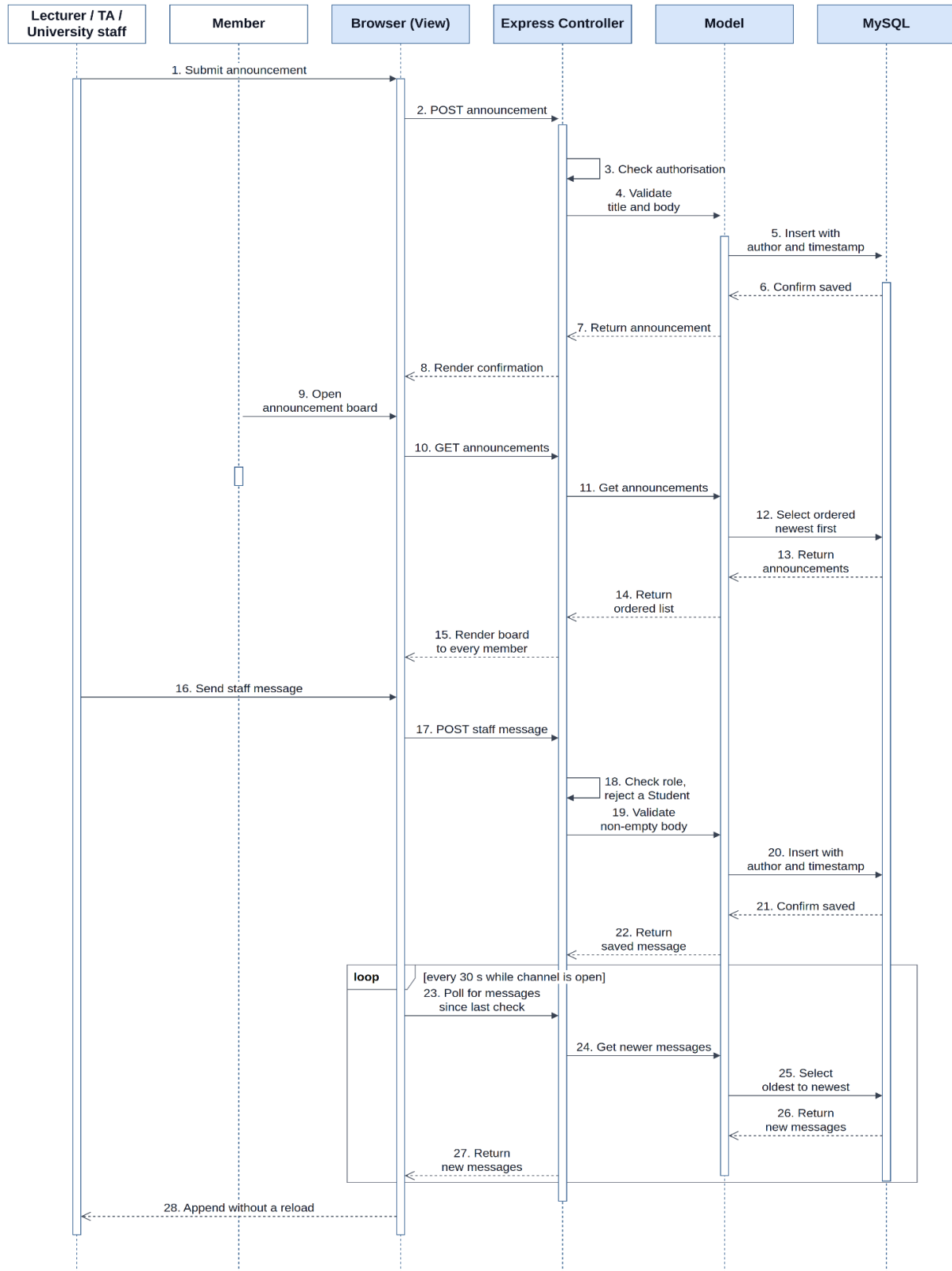
SQD-3: Activity Dashboard, At-Risk Evaluation and Student Schedule

Traced requirements: SRS-12, SRS-13, SRS-14



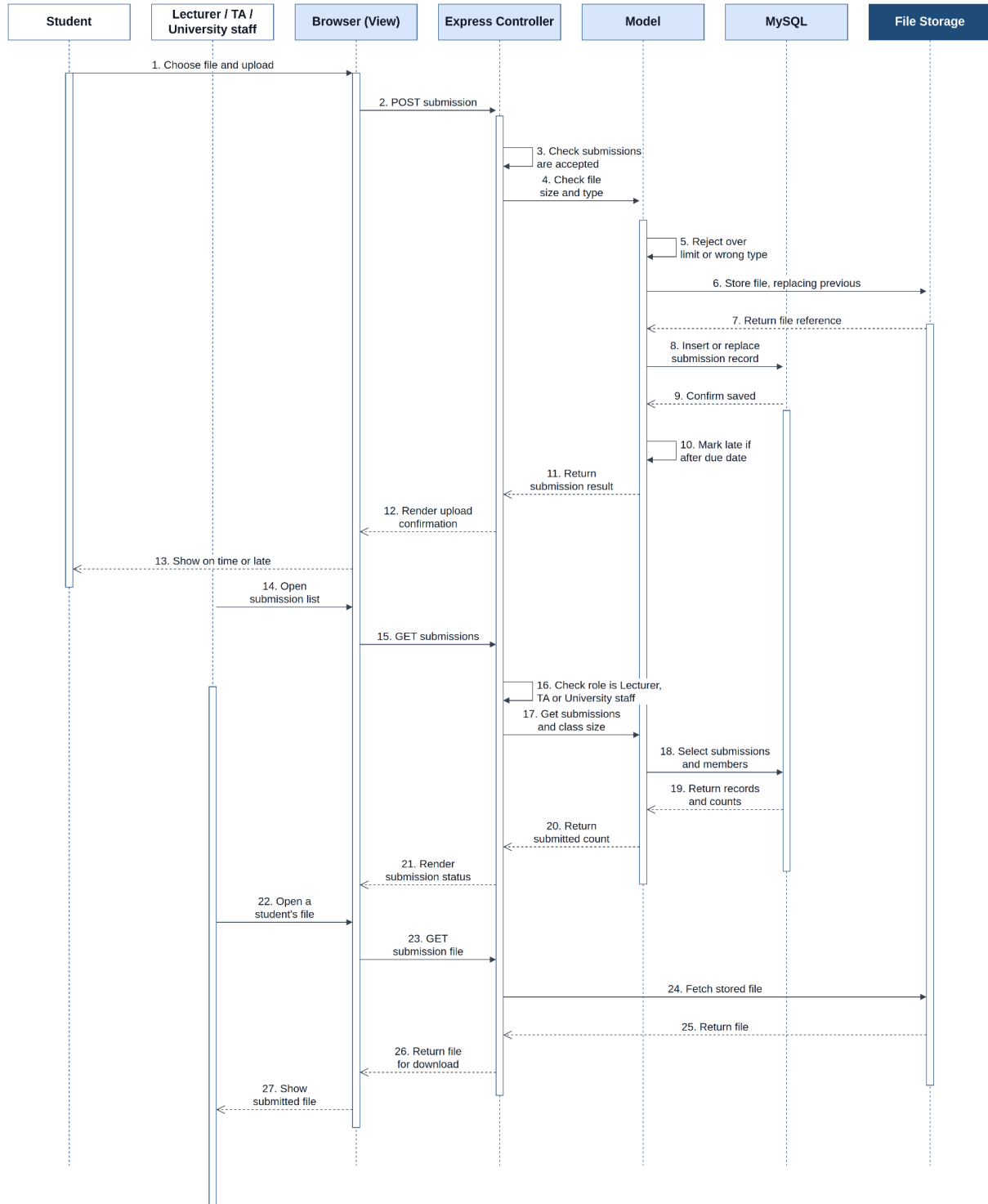
SQD-4: Class Announcements and the Staff Channel

Traced requirements: SRS-15, SRS-16, SRS-17, SRS-18, SRS-19, SRS-20



SQD-5: File Submission and Submission Status

Traced requirements: SRS-21, SRS-22, SRS-23, SRS-24, SRS-25



13. Traceability Matrix

From Sub-Goal to SRS

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	UC-1 Create Classroom, UC-2 Join Classroom by Code, UC-3 Assign Member Role, UC-16 Verify Classroom Membership and Role, UC-17 Authenticate with Google	US-1, US-2, US-3, US-4, US-16	URS-1, URS-2, URS-3, URS-4, URS-16	AD-1, AD-2, AD-3, AD-9	SRS-1 – SRS-7, SRS-26, SRS-27
SG-2	UC-4 Create / Edit Activity, UC-5 Update Activity Status, UC-15 Validate Activity Details	US-5, US-6	URS-5, URS-6	AD-4	SRS-8, SRS-9, SRS-10, SRS-11
SG-3	UC-6 View Activity Dashboard, UC-8 View Class Schedule	US-7, US-9	URS-7, URS-9	AD-4, AD-5	SRS-13, SRS-14
SG-4	UC-7 View At-Risk Activities, UC-13 Submit File, UC-14 View Submission Status	US-8, US-14, US-15	URS-8, URS-14, URS-15	AD-5, AD-8	SRS-12, SRS-13, SRS-21, SRS-22, SRS-23, SRS-24, SRS-25
SG-5	UC-9 Post Announcement, UC-10 View Announcements, UC-11 Post Staff Message, UC-12 View Staff Channel	US-10, US-11, US-12, US-13	URS-10, URS-11, URS-12, URS-13	AD-6, AD-7	SRS-15, SRS-16, SRS-17, SRS-18, SRS-19, SRS-20

From SRS to Sequence Diagram

SRS	SQD
SRS-1	SQD-1
SRS-2	SQD-1
SRS-3	SQD-1
SRS-4	SQD-1
SRS-5	SQD-1
SRS-6	SQD-1
SRS-7	SQD-1 and every protected interaction in SQD-2 to SQD-5
SRS-8	SQD-2
SRS-9	SQD-2
SRS-10	SQD-2
SRS-11	SQD-2
SRS-12	SQD-3
SRS-13	SQD-3
SRS-14	SQD-3
SRS-15	SQD-4
SRS-16	SQD-4
SRS-17	SQD-4
SRS-18	SQD-4
SRS-19	SQD-4
SRS-20	SQD-4
SRS-21	SQD-5
SRS-22	SQD-5
SRS-23	SQD-5
SRS-24	SQD-5
SRS-25	SQD-5